PINKCC

Small talk about
Reproducibility

Chad Estoup--Streiff.

PhD Student at IRCM - PINKCC Lab.

Why having a good level of reproducibility in your code ?

- **Enhances Research Quality:** Open-source code encourages rigorous self-correction and external validation.
- **Boosts Publication Chances:** Many top-tier journals prioritize or require open, reproducible workflows.
- **Increases Citations and Impact:** Making your code accessible allows other researchers to build on your work and cite you.
- **Facilitates Peer Validation:** Open code acts as a stamp of transparency, proving your methodology is sound and reproducible.

Step to make work reproducible

1. **Methodology :** good structure, writing and comments: *Clean code*
2. **Analysis stack:** gold references for Language, libraries, softwares: *Python 3.15, Numpy, Pytorch*
3. **Environment execution:** Cross-plateform and execution consistency: *Docker + UV*

1. Methodology

A. Why it Matters

- **Scientific Transparency:** Code is the "instruction manual" of your math; if it's messy, the manual is unreadable.
- **Reduced Error Rate:** Clean code structure makes bugs easier to spot and harder to hide.
- **Knowledge Transfer:** Ensures that when a PhD student leaves, the lab doesn't lose years of work to "un-runnable" scripts.

1. Methodology

B. Solutions

- **Modularization:** Moving from one 2,000-line main.py to a structured /src folder with dedicated data_processing.py and model_utils.py.

- **Type Hinting:** Using Python’s type system to prevent "Silent Failures" (e.g., passing a String where a Float was expected).

- **Functions documentation:** Each function should be documented with a global description and a precise description of inputs and outputs

- **Auto-Formatters:** Using Black or Ruff to enforce a universal coding style across the entire research team automatically.

Let’s learn and practice with this tutorial

https://testdriven.io/blog/clean-code-python/

2. Analysis stack

A. Why it Matters

- **Community Benchmarking:** Using "Gold Standard" libraries allows other researchers to compare your results against theirs using the same underlying logic.
- **Numerical Stability:** Industry-standard libraries (like NumPy) have been vetted for floating-point errors that custom-written math functions might miss.
- **Future Compatibility:** Popular libraries are maintained for decades, ensuring your code doesn't break six months after publication.

2. Analysis stack

B. Solutions

- **Python 3.15:** Leveraging the latest performance optimizations (like specialized adaptive interpreters).
- **NumPy & PyTorch:** The "Tensor Standard." Using torch.compile for hardware-accelerated research that remains readable.
- **Validation Tools:** Using Pytest to create "unit tests" for your scientific formulas to prove they work before you run the full experiment.
- **Other tools:** just make sure it’s gold standard so other users know how to use it.

3. Environment execution

A. Why it Matters

- **The "Dependency Hell" Cure:** Prevents the "it worked on my laptop" excuse.
- **Bit-for-Bit Reproducibility:** Ensures that X input always equals Y output, regardless of whether it's run on a Mac in 2026 or a Linux server in 2030.
- **Scalability:** Allows you to move your experiment from a local PC to a High-Performance Computing (HPC) cluster instantly.

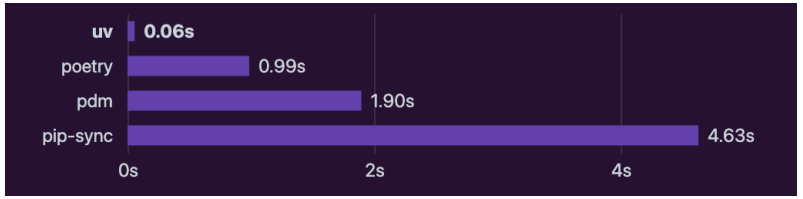
3. Environment execution

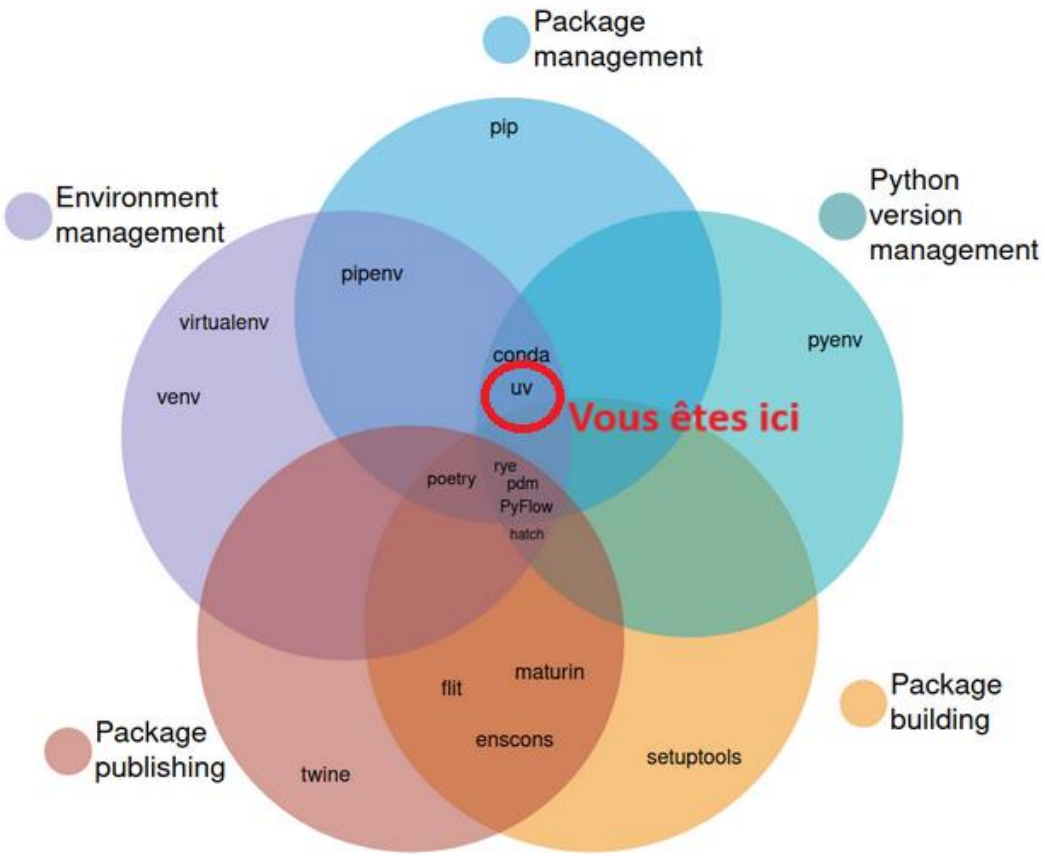
B. Solutions

- **UV (The Modern Standard):** Replacing slow pip installs with a lightning-fast Rust-based resolver. It generates a uv.lock file that freezes your exact environment.
- **Docker:** Wrapping the code, Python version, and system drivers into a Container Image.

What is UV ?

Super fast dependency manager
(thx Rust)



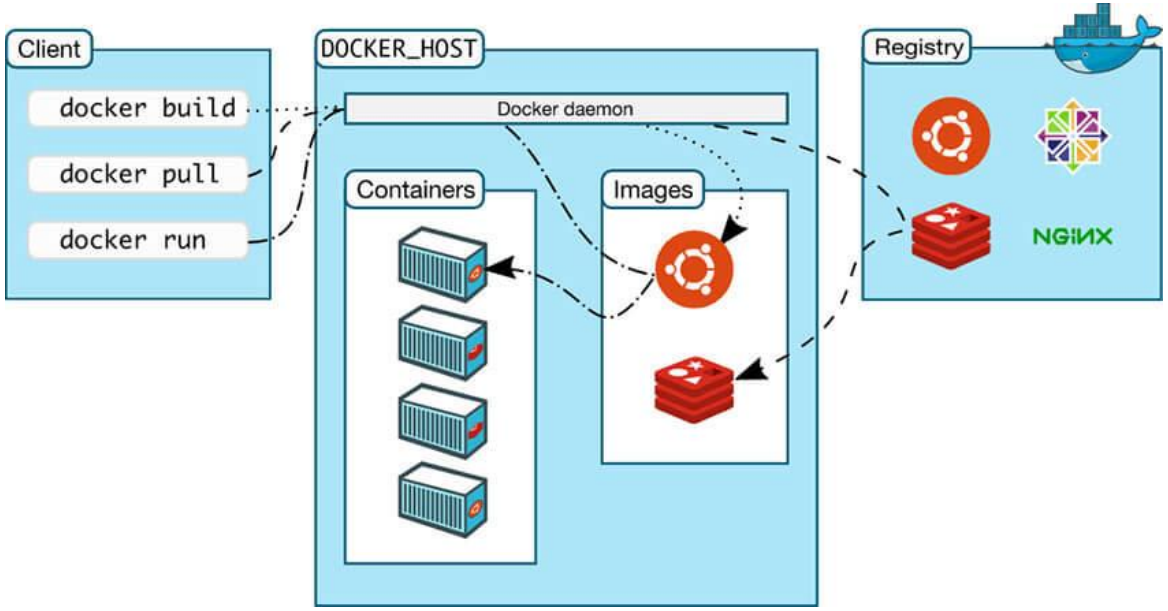


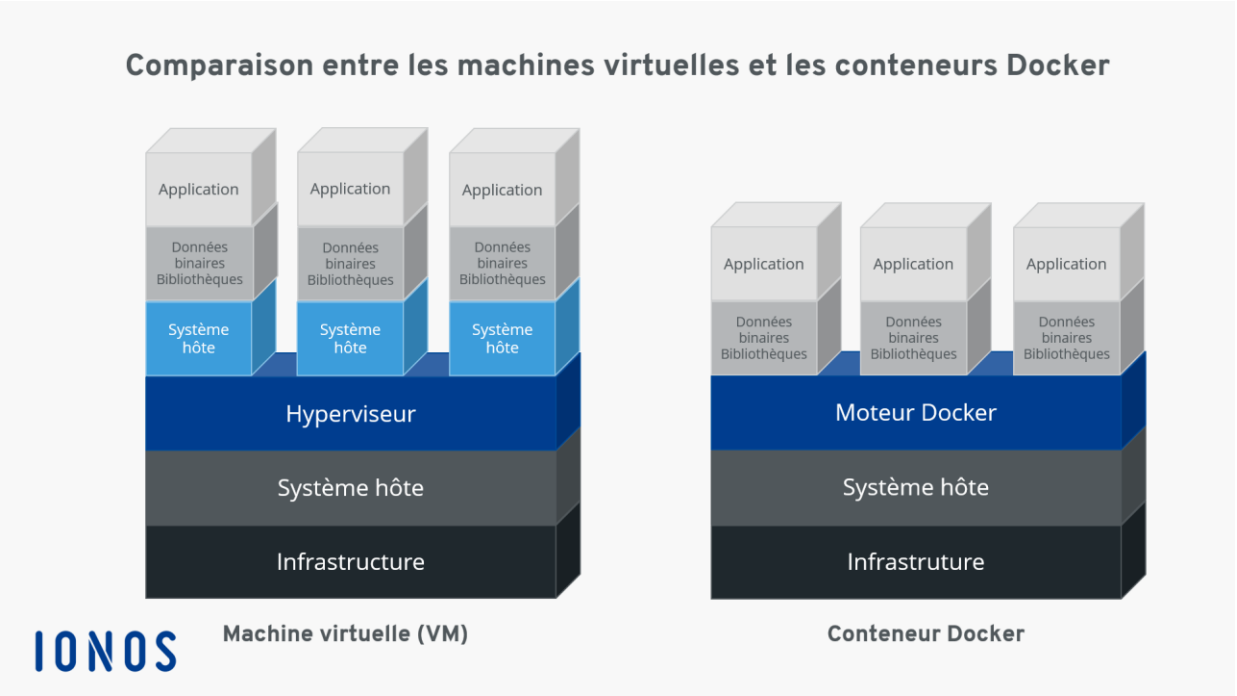
Performance: It is often 10x to 100x faster than pip.

Simplicity: A single tool manages your Python versions, libraries, and virtual environments. Can change python version on the fly.

Reliability: It creates a uv.lock file by default, ensuring your scientific environment is identical every time it’s installed—the "holy grail" of reproducibility.

What is Docker ?





True Reproducibility: It freezes the entire OS environment, ensuring your code behaves exactly the same way in five years as it does today.

Zero Configuration: Collaborators can run your entire research pipeline with one command, without manually installing Python, libraries, or system drivers.

Portability: It eliminates the "it works on my machine" problem, allowing you to move seamlessly from a local laptop to a high-performance computing (HPC) cluster.

The power of the combination of Docker + UV

Feature	uv (The Manager)	Docker (The Container)
Focus	Python libraries and versions	The entire Operating System
Reproducibility	Logical (Lockfiles)	Physical (Images)
Speed	Extremely fast installs	Instant environment launch

The Gold Standard, why this Combo Wins: The "One-Click" Research Pipeline

- **Total Reliability: Docker** freezes the OS, **uv** freezes the libraries. No "it works on my machine" excuses.
- **Future-Proofing:** Your 2026 research stays runnable in 2030. It stops "bit rot" by locking every single dependency in an immutable container.
- **Extreme Speed: uv** removes the "Docker build tax." It installs environments in seconds, making reproducibility a feature, not a chore.
- **Peer Trust:** You move from "Trust me, I did the math" to **"Here is the button to prove it."**

And don't worry, it's very VERY easy 🙇‍♀️

Let’s learn and practice with this tutorial

https://github.com/PINKcc-Lab/docker-uv